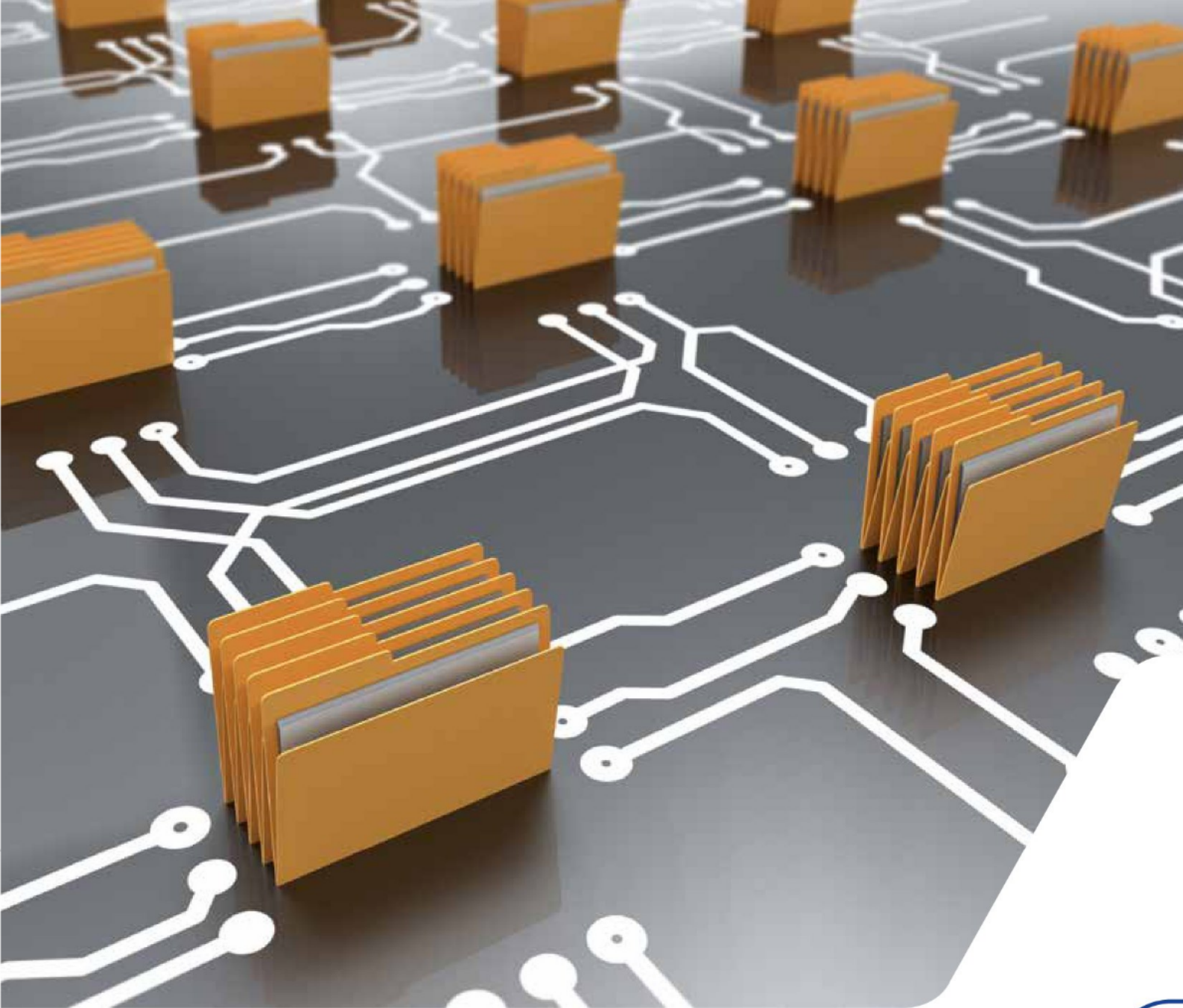




A3 Databases

What are the principles, structures and operations that form the basis of database systems?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.1.1 Explain the features, benefits and limitations of a relational database

A3.1.1 Relational databases

■ Features

◆ **Database:** an organized collection of structured information or data that can be accessed in different ways.

◆ **Table:** a structure of rows and columns for storing a group of similar data.

◆ **Entity:** a living or non-living thing that can have data stored about it that can be described, e.g. a person, a chair or an aeroplane.

◆ **Tuple:** one instance of an entity; a row in a table.

◆ **Record:** one instance of an entity; a row in a table.

◆ **Attribute:** a data item or a characteristic of an entity; a column in a table.

A **database** refers to an organized collection of structured information or data that can be accessed in different ways. Typically, a database is stored electronically for fast retrieval and manipulation of data. There are different types of databases, but your focus will be on relational databases.

Relational databases have been predominantly used since 1980. Data in a relational database is organized as a set of tables made of columns and rows.

Tables

The **table** is used to describe entities. An **entity** refers to a living or non-living thing that can have data stored about it that can be described, such as a person, a chair or an aeroplane.

Each row, also called a **tuple**, will include a **record** or an instance of an entity, for example a specific person, a specific chair or a specific aeroplane. Each column, also called a “field” or an **attribute**, will include a data item or a characteristic of an entity, such as the age or the name of a person, the colour of a chair, the model of an aeroplane, and so on.

Consider the following example:

AEROPLANE

Model	Manufacturer	PhysicalClassEngine	NoOfEngines
Rockwell Commander 112	Rockwell	Piston	1
Airbus A319 Neo	Airbus	Jet	2
Boeing 747-100	Boeing	Jet	4
Boeing 777-8	Boeing	Jet	2
Airbus A400M Atlas	Airbus	Turboprop	4
Boeing 747-100	Boeing	Jet	4

In the example above, the table name is AEROPLANE (this is the entity) and there are six records or tuples (rows in the table, excluding the table heading) and four attributes or fields (columns).

Primary key

If you are to uniquely identify a record in this table, you should add an extra field. This is required, as all the given fields have repeating values, so they cannot be used to identify a record. It is possible for a company to have two aeroplanes of the same type, manufactured by the same company, with the same type of engine, and so on.

Therefore, by adding an extra field to uniquely identify each record, the table will look like this:

AEROPLANE

PlaneID	Model	Manufacturer	PhysicalClassEngine	NoOfEngines
A01	Rockwell Commander 112	Rockwell	Piston	1
A02	Airbus A319 Neo	Airbus	Jet	2
A03	Boeing 747-100	Boeing	Jet	4
A04	Boeing 777-8	Boeing	Jet	2
A05	Airbus A400M Atlas	Airbus	Turboprop	4
A06	Boeing 747-100	Boeing	Jet	4

◆ **Primary key:** a field that uniquely identifies a record in a table.

◆ **Foreign key:** an attribute in a table that refers to the primary key in another table.

The PlaneID field is unique for each record (it has no duplicates) and this is called a **primary key**.

Foreign key

Consider that you want to record data about specific flights at an airport. The AEROPLANE table only provides information about the planes. Therefore, you will need to create a new table to register the flight details.

FLIGHTS

FlightID	Departure	Destination	PlaneID	FlightDate	DeptTime	ArrivalTime
LG8903	LUX	OTP	A03	01/07/25	17:00	20:20
OS864	CAI	VIE	A04	15/01/25	16:45	19:20
GB961	LHR	ZRH	A03	25/02/25	8:40	11:35

In this table, the FlightID acts as a primary key.

The two tables, FLIGHTS and AEROPLANE, are building a relationship, as the PlaneID from the AEROPLANE table is used in the FLIGHTS table to identify the type of plane being used for a specific flight. However, the PlaneID is no longer a primary key in the FLIGHTS table, as it has repeating values; here it acts as a **foreign key**.

A foreign key is an attribute or a set of attributes in one table that refers to the primary key in another table.

Composite key

If you are to introduce a third table to register pilots on the flight, it might look like this:

PILOTFLIGHT

FlightID	PilotID
LG8903	P500
OS864	P104
GB961	P500

◆ **Composite key:** a set of attributes that form a primary key.

◆ **Relationship:** a relation established between different tables, where the foreign key in one table refers to the primary key in another table.

The `FlightID` links to the `FLIGHTS` table and the `PilotID` links to the `PILOTS` table (supposing there is a `PILOTS` table as well that records pilots' details). In the `PILOTFLIGHT` table, there is no primary key. A solution could be to use a **composite key**, formed from the two attributes `FlightID` and `PilotID`.

A composite key is a set of attributes that forms a primary key to provide a unique identifier for a table.

■ Relationships

A **relationship** is created when there is a logical association between two or more database tables, in which one table contains one or more foreign keys that reference the primary keys of the other tables. They enable relational databases to divide and store data in separate tables, while connecting their data items.

To ensure data is always accurate, accessible and consistent, relational databases follow certain integrity rules. For example, the referential integrity rule prevents users or applications from entering inconsistent data. It is a constraint that ensures that no table will contain values of a foreign key that are not matched to the corresponding primary key. In other words, it makes sure that a foreign key always refers to a record that exists in another table. By applying referential integrity constraints, the data stays consistent throughout operations such as insertion, deletion and modification of tuples.

There are several types of relationships:

- one-to-one (1:1)
- one-to-many (1:m)
- many-to-one (m:1)
- many-to-many (m:m).

One-to-one relationships

When there is a one-to-one relationship between two tables, that means that one record in a table is associated with exactly one record in another table: the primary key corresponds to one or no data in another table. For example, each staff member of a school has one single staff ID; each country has exactly one capital city; or a user on a social media platform has a single user profile. Those are very rare types of relationships, which you will not frequently encounter when dealing with databases.

One-to-many relationships

This is a frequently used type of relationship, and it refers to one record in a table being associated with one or more records in another table: the foreign key of one table references the primary key of another table. Examples of one-to-many relationships are where one teacher teaches many subjects; one tourist visits many countries; one person owns many properties; one person has many bank accounts.

Many-to-one relationships

Many-to-one relationships are similar to one-to-many relationships, but they differ in their directionality. The availability of the entity and the side of the relationship it is on determines whether it is a one-to-many or a many-to-one relationship. For example, if one teacher is teaching multiple subjects, the relationship between the teachers and the subjects is one-to-many, while the relationship between subjects and the teachers is many-to-one. Examples of many-to-one relationships are where many students enrol in a single course; many people work for a single company; there are many galaxies in the universe.

Many-to-many relationships

This type of relationship appears when multiple records in a table have a relation with multiple records in another table. Examples of many-to-many relationships are where many customers purchase many products; many actors act in many movies.

The problem with a many-to-many relationship is that a foreign key attribute can hold a single value and so it cannot handle the many references required.

To implement such relationships in relational databases you must introduce a linking entity. This means that two one-to-many relationships will be created: one between the first table and the linking table and another one between the second table and the linking table.

In the example above, when you wanted to connect the `FLIGHTS` table with the `PILOTS` table, a third table was introduced called `PILOTFLIGHT`. As such, a relationship of one-to-many was established between the `FLIGHTS` and `PILOTFLIGHT` tables and a relationship of one-to-many was established between the `PILOTS` and `PILOTFLIGHT` tables. This is done because a many-to-many relationship cannot be physically represented in a database.

■ Benefits of relational databases

Community support

Relational databases have been around since the 1970s, and this is the most widely accepted model for databases. Therefore, there are lots of online communities able to provide support and guidance in building, maintaining and troubleshooting them.

Concurrency control

Concurrency control is a crucial database management system (DBMS) component. It manages simultaneous operations without them conflicting with each other, and its purpose is to maintain data integrity, consistency and isolation when multiple users or applications access the database at the same time.

Data consistency

Data consistency refers to data remaining in a consistent state from start to finish, reinforcing data integrity. This means that all copies or instances of the data are the same across all systems and databases. In relational databases, each piece of data is stored in only one place, and all related data is stored together in the same table. This ensures all users have access to accurate and up-to-date information.

Data integrity

Data integrity refers to the accuracy, completeness and consistency of data throughout its lifecycle. It ensures the data hasn't been tampered with or altered in any unauthorized way. Data validation techniques can be used to ensure data integrity.

Data retrieval

The process of retrieving data from a relational database is efficient and flexible. SQL allows for complex queries to be written to retrieve exactly the data needed, using `SELECT` statements, `JOINS`, `WHERE` clauses, and more. Users can also create ad hoc queries to retrieve data without needing predefined reports or programs.

Reduced data duplication

Relational databases ensure that you have common fields to be used to link up tables and match records, without having to duplicate all the details several times. Identifying and removing duplicate data reduces the amount of storage needed to store the data.

Reduced redundancy

Data redundancy refers to storing the same data in multiple locations at the same time. This may lead to inconsistencies, partial updates and unnecessary duplications. Relational databases allow you to reduce redundancy by normalizing the database (organizing the data to be stored into several tables, creating relationships between them to avoid repeated groups of attributes, and correctly enforcing their dependencies; non-key attributes being independent).

Reliable transaction processing

A transaction refers to a sequence of actions performed on a database that is considered as a single unit (such as inserting, deleting, updating data in a table). A transaction is a unit of work, or a logical action, that is independent of other transactions and is performed on a database by a database management system. A transaction is either executed in full or it is not executed at all. Transactions ensure data integrity and reliability within relational databases.

Scalability

Database scalability refers to the ability of a database to handle increasing amounts of data, number of users and types of requests without sacrificing performance or availability.

Relational databases are vertically scalable, meaning that they support the idea of adding more resources (CPU, RAM, hard drive space) to existing systems, which is a cheaper, easier and faster approach to handling increases.

Security features

Relational databases increase security by controlling access to stored data, ensuring only authorized users can interact with the database. They allow the assignment of unique user accounts with specific permissions based on the users' roles and responsibilities. They allow different views of tables for different access rights.

■ Limitations of relational databases

Big data scalability issues

Relational databases can be more difficult to scale as the size and complexity of the data increases. The performance can drop when manipulating large data sets (horizontal scaling) or dealing with complex queries; joins between tables can be slow and indexing strategies can be difficult to optimize.

Design complexity

Relational databases require a lot of structure and planning to design the tables and the relationships between them in a way that fits correctly to the requirements.

Hierarchical data handling

Storing hierarchical data in relational databases is challenging due to the mismatch between the hierarchical structure and the tabular nature of relational databases. Even if this is done through a strategy such as an adjacency list model (where each record contains a reference to its parent record, forming a tree-like structure, such as an employee table having a field referencing the manager's ID for each employee), it is challenging to retrieve and traverse hierarchies, especially for large sets of data, or to reorder nodes and perform queries on the subtrees created.

Rigid schema

Relational databases have a predefined schema (structure of the data and how it will be stored in the database). Defining the schema can be challenging as it is not easy to predict the data structure of the database beforehand, and changing it later is complicated. When it comes to changing the database structure, updating the schema is time-consuming and complicated.

Object-relational impedance mismatch

Object-relational impedance mismatch refers to the difficulties encountered when relational databases are used by a program written in an object-oriented programming language. A major mismatch between relational databases and OOP languages is the data type differences. Relational models do not allow the use of by-reference attributes (pointers), while OOP languages embrace this behaviour. There is no clear way to translate all OOP concepts into relational databases or vice versa, such as there is no way to translate inheritance to a relational database concept.

Unstructured data handling

Unstructured data refers to a collection of data where one record differs from another record. Not being able to identify common fields or attributes for the records makes it impossible to design a schema for such data (to represent them as relational databases).

Common mistake

When asked to explain concepts such as benefits and limitations of relational databases within a given scenario, candidates often identify general benefits and limitations without making any connections to the given scenario. To gain full marks, the scenario must be taken into consideration, as well as the number of marks awarded for the respective question.

ACTIVITY

Self-management skills: Create plans to prepare for summative assessments – keep track of topics that have been covered and how well you mastered each of them. Identify what you can do to master any topics you found more challenging.

REVIEW QUESTIONS

- 1 Define the terms:
 - a “primary key”
 - b “foreign key”.
- 2 Explain what is meant by referential integrity.
- 3 Explain the one-to-one, one-to-many and many-to-many types of relationships.
- 4 Discuss the benefits and limitations of a relational database.
- 5 Define the term “database”.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.2.1 Describe database schema
- ▶ A3.2.2 Construct entity-relationship diagrams (ERDs)
- ▶ A3.2.3 Outline the different data types used in relational databases
- ▶ A3.2.4 Construct tables for relational databases
- ▶ A3.2.5 Explain the difference between normal forms
- ▶ A3.2.6 Construct a database normalized to 3NF for a range of real-life scenarios
- ▶ A3.2.7 Evaluate the need for denormalizing databases

A3.2.1 Database schema

◆ **Database schema:** an architecture showing how data is organized and how the relationship between data is managed.

◆ **Conceptual schema:** an abstract model describing the structure of the data without considering how it will physically be implemented.

Database schema is an architecture showing how data is organized and how the relationship between data is managed. It provides a logical view of the database.

There are different types of database schemas:

- **Conceptual schema:** An abstract model describing the structure of the data without considering how it will physically be implemented.
- **Logical schema:** A detailed design of the structure of the tables (fields and data types), relationships between tables and constraints.
- **Physical schema:** Represents the implementation of the logical schema into a specific DBMS (database management system), showing how data is stored, indexed or accessed.

The use of database schemas improves:

- **data organization:** it provides clear structure for storing and organizing the data
- **data security:** it defines user permissions and views to protect data
- **data integrity:** it uses rules and constraints to maintain data accuracy and consistency
- **performance:** through the use of queries
- **scalability:** it allows for changes to the database without disrupting current applications.

The DBMS controls the creation, maintenance and usage of a database and it mediates between the data-handling applications and the operating system. The DBMS offers features such as database queries, forms, reports and charts to display the data.

■ Conceptual schema

Conceptual schema is a high-level representation of the database, defining its structure and organization. It is an abstract model that hides details such as implementation of the data structures or physical storage. It defines the entities, attributes and relationships between entities.

A common method of implementing conceptual schema is by using entity-relationship diagrams (ERDs).

For example, consider a sales system with the following structure:

- Entities
 - Products
 - Orders
 - Customers
- Attributes
 - In Products (ProductID, ProductName, Price)
 - In Orders (OrderID, OrderDate)
 - In Customers (CustomerID, CustomerName, EmailAddress)
- Relationships
 - Customer places an order
 - An order includes one or more products

Conceptual schema is a model with insufficient details to build an actual database.

■ Logical schema

◆ **Logical schema:**
a detailed design of
the structure of tables
(fields and data types),
relationships between
tables and constraints.

Logical schema is a model that defines the structure of the database, including entities, attributes, data types, constraints, keys and relationships. It is a design that doesn't take into consideration the requirements of a specific database management system (DBMS). The logical schema is derived from the conceptual schema by:

- converting the entities into detailed tables
- defining the attributes by specifying the data types and constraints for each field in the table
- establishing primary and foreign keys
- defining relationships between the tables by using the keys
- normalizing the database to minimize data redundancy
- ensuring data integrity.

In the previous example:

Tables:

- Products
 - ProductID: INTEGER (PRIMARY KEY)
 - ProductName: VARCHAR
 - Price: REAL
- Orders
 - OrderID: INTEGER (PRIMARY KEY)
 - OrderDate: DATE
 - CustomerID: INTEGER (FOREIGN KEY)
 - PRODUCTID: INTEGER (FOREIGN KEY)
- Customers
 - CustomerID: PRIMARY KEY
 - CustomerName: VARCHAR
 - EmailAddress: VARCHAR, UNIQUE

Relationships:

- A customer places one or more orders (one-to-many).
- An order includes one or more products (one-to-many).

◆ **Physical schema:** an implementation of logical schema into a specific DBMS (database management system), showing how data is stored, indexed or accessed.

■ Physical schema

Physical schema includes specifics of storage devices, access methods, indexing, partitioning, access methods, views and configuration of the database on the storage media. It translates the logical schema into an implementation that fits the requirements of a specific database management system.

In the previous example:

Tables:

■ Products

- ProductID: INT PRIMARY KEY AUTO_INCREMENT
- ProductName: VARCHAR(100) NOT NULL
- Price: REAL NOT NULL

INDEX on ProductName for faster access based on the product name

■ Orders

- OrderID: INT PRIMARY KEY AUTO_INCREMENT
- OrderDate: DATE NOT NULL
- CustomerID: INT FOREIGN KEY NOT NULL
- ProductID: INT FOREIGN KEY NOT NULL

INDEX on CustomerID and ProductID for faster joins

■ Customers

- CustomerID: INT PRIMARY KEY AUTO_INCREMENT
- LastName: VARCHAR(100) NOT NULL
- FirstName: VARCHAR(100) NOT NULL
- EmailAddress: VARCHAR(100) NOT NULL UNIQUE

INDEX on LastName for faster access based on the last name

Storage parameters:

- Use indices described above for fast retrieval of data.
- Partition large tables like Orders by OrderID to improve query performance.

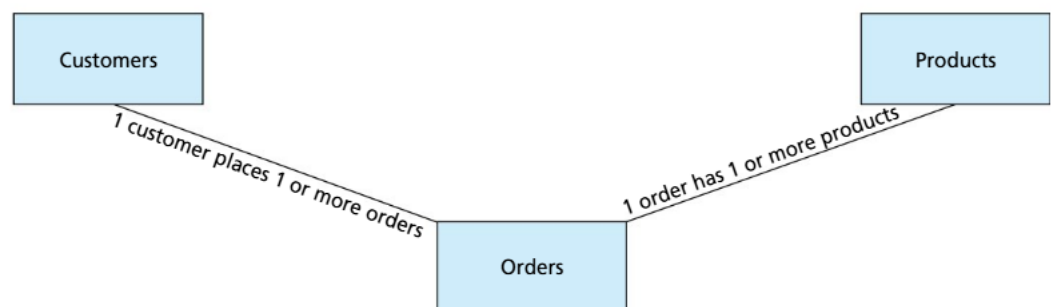
A3.2.2 Entity-relationship diagrams

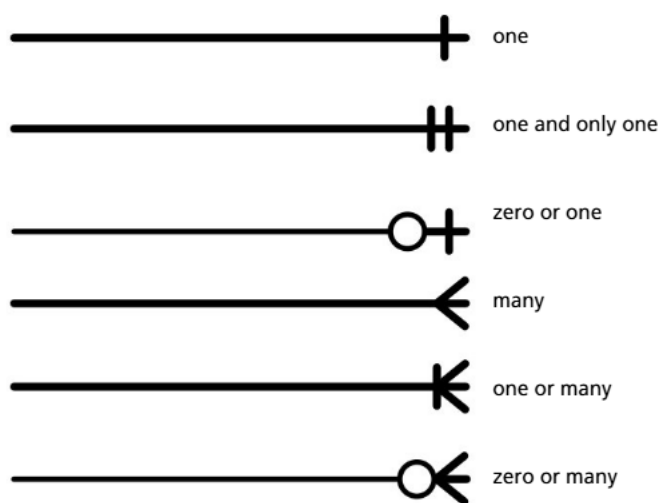
◆ **Entity-relationship diagram:** a visual representation of the entities in a database and the relationship between them.

An **entity-relationship diagram** (ERD) is a visual representation of the entities in the database and the relationship between them.

Besides providing a clear overview of the database structure, ERDs facilitate communication between stakeholders; act as documentation for the database design; support future development and maintenance of the database; and ensure data integrity and consistency through constraints and well-defined relationships.

For the sales system with Products, Orders and Customers entities, the ERD looks like this:





■ Modality of relationships

◆ **Modality:** the minimum number of instances of one entity that can be associated with an instance of another entity.

◆ **Cardinality:** the maximum number of times an instance in one entity can be associated with instances in the related entity.

The cardinality of relationships refers to the nature and extent of relationships between entities in an ERD. It specifies the number of instances of one entity that can or must be associated with each instance of another entity.

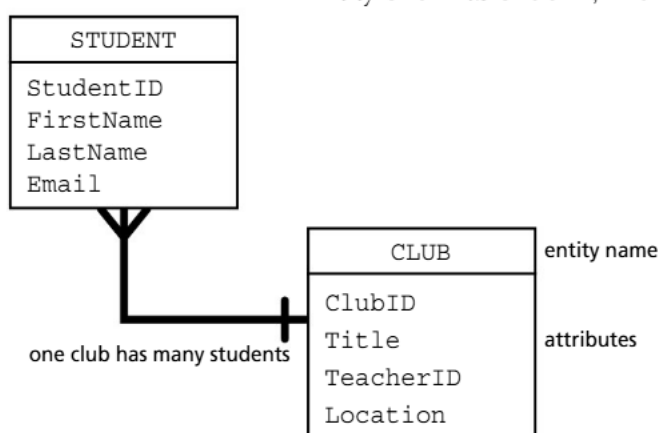
Cardinality refers to the maximum number of times an instance in one entity can be associated with instances in the related entity. It describes the “many” side of the relationship and it can be defined as:

- one-to-one
- one-to-many
- many-to-one
- many-to-many.

For example, consider a school management system that includes students and clubs as entities.

Entity STUDENT has StudentID, FirstName, LastName, Email as attributes.

Entity CLUB has ClubID, Title, TeacherID, Location as attributes.



Modality in ERDs refers to the minimum number of instances of one entity that can be associated with an instance of another entity. It defines whether the participation of an entity in a relationship is optional (0) or mandatory (1).

Consider an example involving data about patients and their medical records in a medical healthcare system. Most patients will have associated medical records, but new patients or newborn children might not have any medical history, therefore this is a type of optional relationship.

On the other hand, if you are to consider an e-commerce platform, every order must be associated with a customer (you cannot become a customer unless you place an order), so that is a type of mandatory relationship.

The relationship between the two entities can be represented as “a club has many students”.

Cardinality: one student can enrol in multiple clubs (one-to-many).

Modality: a club must have at least one student enrolled (mandatory for clubs); a student might not enrol in any clubs (optional for students).

Understanding both cardinality and modality is essential for accurately modelling the relationships and constraints in a database, ensuring it effectively reflects the real-world requirements and business rules.

ACTIVITY

Social skills: Support other students with application skills on practical tasks – help your classmates to analyse different database scenarios, identify entities, establish the appropriate relationships between tables and provide feedback to each other on how appropriate ERDs can be created.

A3.2.3 Data types used in relational databases

Data type for attributes	Description
CHARACTER	Fixed length text
VARCHAR(n)	Variable length text (n indicates the maximum number of characters)
INTEGER	Whole number
REAL	Number with a decimal part
DATE	Date as YYYY-MM-DD
TIME	Time as HH:MM:SS
BOOLEAN	True or False

Choosing the right data type is important for ensuring efficient indexing. For example, using `CHARACTER(8)` for fixed-length data like `UserID` is more efficient than using `VARCHAR(8)`, as this can lead to extra time during query execution due to the variable length storage. Also, the data type indicates the type of operations permitted. For example, if you store the quantity and price as fixed-length text, to perform calculations you will need to convert the text to integer or real values in the application, before using the data.

Another aspect of using appropriate data types is being able to store the data in the database into the corresponding field. If the type of data does not match the data type of the attribute in the database, the insertion attempt will throw errors.

Data consistency ensures users have access to up-to-date and accurate information, where all copies or instances are the same across all systems and database tables. Using different data types to refer to the same attribute on different platforms (database and application system) will lead to problems such as not being able to perform operations specific to the required data type, incorrect updates or queries.

A3.2.4 Constructing tables for relational databases

Properly defining the tables in a database supports the design of appropriate ERDs and ensures data integrity.

Considering a school management system, this could include the following tables:

■ `STUDENT (StudentID, FirstName, LastName, DateOfBirth, Email)`

■ `CLUB (StudentID, ClubTitle, TeacherName)`

■ `TEACHER (TeacherClub, Location)`

`STUDENT`

StudentID	FirstName	LastName	DateOfBirth	Email
101	Fatema	Kada	02/01/2010	f.kada@email.com
105	Alexandru	Buchidau	05/11/2009	a.buchidau@email.com
202	Kada	Hussein	07/25/2011	k.hussein@email.com

In the `STUDENT` table, the `StudentID` acts as a primary key to uniquely identify each record in the table.

CLUB

StudentID	ClubTitle	TeacherName
105	Robotics	Bobby Williams
202	Taekwondo	Dima White
101	Robotics	Bobby Williams
105	Arts and Crafts	Jane Doe

In the CLUB table, the StudentID is a foreign key (as it is a primary key in the STUDENT table). However, none of the fields in this table can act as a primary key, as they all have duplicates. But you could set the primary key to be a composite key, formed from the attributes StudentID and ClubTitle. On the other hand, you could add a new attribute ClubID to act as a primary key. In case there is a need for a single field to act as primary key, it is possible to combine data from several attributes into one to act as a concatenated key.

TEACHER

TeacherClub	Location
Jane Doe Arts and Crafts	L101
Bobby Williams Robotics	H203
Dima White Taekwondo	B353

In the TEACHER table, the primary key is a concatenated key, formed from the attributes TeacherName and ClubTitle.

A3.2.5 Differences between normal forms

◆ **Normalization:** the process of organizing data in a relational database in a way to reduce data redundancy and to improve data integrity.

◆ **First normal form:** the status of a relational database in which entities do not contain repeating groups of attributes.

◆ **Atomic:** each attribute in a table containing indivisible values (values that cannot be broken down into more detailed sub-values).

Data **normalization** represents the process of organizing data in a relational database in a way to reduce data redundancy and to improve data integrity. Data redundancy is reduced as each item of data only occurs in one location in the database. This can reduce the possibility of update anomalies occurring, and it makes more efficient use of memory. Normalization leads to smaller tables with less information in each row, which leads to a reduction of input / output transfers, and so the CPU can work at full capacity since the likelihood of CPU activities being suspended is reduced. Normalization is achieved through a series of stages called “normal forms”, where each normal form has specific requirements for the table to be considered normalized at that level.

■ First normal form (1NF)

In **first normal form**, the table:

- has a primary key
- includes no duplicate attributes from the same table
- includes no repeated groups of attributes.

Therefore, you need to create separate tables for each group of related data, identifying each record by using the primary key, which is made of one single attribute or a set of attributes (composite or compound key), and ensure the entities do not contain repeated groups of attributes.

In 1NF, data in each field must be **atomic**. This means that each attribute contains indivisible values (values that cannot be broken down into more detailed sub-values). For example, an attribute called TeacherName in the TEACHER table is not an atomic field as this could

◆ **Functional dependency:** a relationship that exists between attributes, where one set of attributes (the determinant) determines the value of the other set (the dependent).

◆ **Full functional dependency:** where dependent attributes are determined by the determinant attributes.

◆ **Partial functional dependency:** when dependent attributes are partially determined by the determinant attributes.

◆ **Transitive dependency:** a type of functional dependency that occurs when a non-prime attribute is dependent on another non-prime attribute, rather than on the primary key.

◆ **Second normal form:** the status of a relational database in which entities are in 1NF and any non-key attributes depend upon the primary key.

◆ **Third normal form:** the status of a relational database in which entities are in 2NF and all non-key attributes are independent.

be further split into two different attributes called `LastName` and `FirstName`. Once this is achieved, the fields are atomic.

Atomicity ensures that each cell in the table will contain a single value, not complex structures like arrays or lists.

Functional dependency is a relationship that exists between attributes, where one set of attributes (the determinant) determines the value of the other set (the dependent). Typically, this is a relationship between the primary-key attribute and a non-key attribute. For example, in the `STUDENT` table, the `StudentID` (primary key and the determinant) determines the `FirstName`, `LastName`, `DateOfBirth` and `Email` values (the dependent). This means that, given the value of the `StudentID`, you can find the other details, but not vice versa. To ensure functional dependency in 1NF, you need to ensure entity atomicity and to remove repeating groups of attributes.

There are different types of functional dependencies:

- **Full functional dependency:** The dependent attributes are determined by the determinant attributes. For example, the `StudentID` fully determines the student's `FirstName`, `LastName`, `DateOfBirth` and `Email`.
- **Partial functional dependency:** The dependent attributes are partially determined by the determinant attributes. For example, the `StudentID` could partially determine the `FirstName`, `LastName` and `DateOfBirth` of the student, but not their course instructor for a club.
- **Transitive dependency:** The dependent attributes are determined by a set of attributes that are not included in the determinant attributes. For example, in an `EMPLOYEES` table, the `EmployeeID` may determine the `EmployeeDepartment`, which in turns determines their salary.

■ Second normal form (2NF)

In **second normal form** (2NF):

- entities are in 1NF
- any non-key attributes are fully functionally dependent on the primary key; there are no partial dependencies.

Partial-key dependency occurs in a table that has a composite key as primary key and one or more non-key attributes are dependent on only a subset of the composite primary key, rather than on the entire composite key. For example, in the `CLUB` table, the non-key attribute `TeacherName` is dependent on the `ClubTitle`, but not on the `StudentID`. As the primary key in this table is a composite key formed of both the `ClubTitle` and the `StudentID` fields, the `TeacherName` should have been fully functionally dependent on these two fields.

■ Third normal form (3NF)

In **third normal form** (3NF):

- entities are in 2NF
- all non-key attributes are independent (remove columns that are not fully functionally dependent on the primary key); the table contains no non-key dependencies.

Non-key or transitive dependency is a type of functional dependency that occurs when a non-prime attribute is dependent on another non-prime attribute, rather than on the primary key.

If the CLUB table looked like the one below, the primary key in the table would be ClubID. The ClubTitle is fully functionally dependent on the ClubID; however, the TeacherLastName is dependent on the TeacherID, which is not a primary key in the table.

CLUB

ClubID	ClubTitle	TeacherID	TeacherLastName
105	Robotics	1	Williams
202	Taekwondo	2	White
105	Robotics	1	Williams
106	Arts and Crafts	4	Doe

To resolve this non-key dependency, the table should be split into two: one storing club details (ClubID, ClubTitle and TeacherID) and the other teacher details (TeacherID and TeacherLastName).

CLUBDETAILS

ClubID	ClubTitle	TeacherID
105	Robotics	1
202	Taekwondo	2
106	Arts and Crafts	4

TEACHERDETAILS

TeacherID	TeacherFirstName	TeacherLastName
1	Bobby	Williams
2	Dima	White
4	Jane	Doe

Normalization issues can encompass data duplication, missing data and a range of dependency concerns, including data dependencies, composite key dependencies, transitive dependencies and multi-valued dependencies. For example, a car manufacturer produces two colours (black and grey) of each model every year. The attributes Colour and ManufacturingYear are dependent on the field CarModel, but they are independent of each other. Therefore, they can be called “multi-valued dependencies” on the CarModel.

Multi-valued dependencies occur when two attributes in a table are independent of each other, but both depend on a third attribute. This is important for achieving fourth normal form (4NF), which addresses certain types of redundancy not handled by earlier normal forms.

A3.2.6 Normalized databases (3NF)

Consider a library management system that stores the data in a table called “books”:

BOOKS

BookID	AuthorID	Author	Title	Pages	ProofReader
1	101	Boris Brown	History of AI	353	Amanda
2	102	Chris Joe	The Great G	200	Hamilton
3	19	Danny Bill	Big Tonny	190	Juan
5	101	Boris Brown	Amazing Future	399	Amanda

Normalizing this database to 3NF means:

1 Normalize it to 1NF:

- ☐ Set BookID as the primary key.
- ☐ Split the author into two different attributes: AuthorFirstName and AuthorLastName.

BookID	AuthorID	AuthorFirstName	AuthorLastName	Title	Pages	ProofReader
1	101	Boris	Brown	History of AI	353	Amanda
2	102	Chris	Joe	The Great G	200	Hamilton
3	19	Danny	Bill	Big Tonny	190	Juan
5	101	Boris	Brown	Amazing Future	399	Amanda

2 Normalize it to 2NF:

- ☐ Entities are in 1NF.
- ☐ There are no partial dependencies.

AuthorFirstName and AuthorLastName are dependent on AuthorID, while Title, Pages and ProofReader are dependent on the primary key (BookID). Therefore, we need to split this table as follows:

BOOKS

BookID	AuthorID	Title	Pages	ProofReader
1	101	History of AI	353	Amanda
2	102	The Great G	200	Hamilton
3	19	Big Tonny	190	Juan
5	101	Amazing Future	399	Amanda

AUTHOR

AuthorID	AuthorFirstName	AuthorLastName
101	Boris	Brown
102	Chris	Joe
19	Danny	Bill

3 Normalize it to 3NF:

- ☐ Entities are in 2NF.
- ☐ There are no transitive dependencies.

The ProofReader field has repeating values, and it is not necessarily fully functionally dependent on the BookID. Therefore, to remove non-transitive dependencies, you can create a new table for proof readers.

BOOKS

BookID	AuthorID	Title	Pages
1	101	History of AI	353
2	102	The Great G	200
3	19	Big Tonny	190
5	101	Amazing Future	399

AUTHOR

AuthorID	AuthorFirstName	AuthorLastName
101	Boris	Brown
102	Chris	Joe
19	Danny	Bill

PROOFREADERS

ProofReaderID	ProofReader
100	Amanda
222	Hamilton
123	Juan

Top tip!

Normalization of databases may be a challenging concept to grasp. Take the time to practise this concept; you can use past papers' exercises or create your own tables for this purpose.

Now, you need to link the BOOKS table with the PROOFREADERS table, so a new table is created.

BOOKS_PROOFREADERS

BookID	ProofReaderID
1	100
2	222
3	123
5	100

A3.2.7 Denormalizing databases

There are both advantages and disadvantages to normalizing and denormalizing databases. Overall, normalization plays a crucial role in designing efficient, maintainable and reliable databases that support data integrity and consistency while optimizing performance and scalability.

Normalization

Advantages	Disadvantages
Minimizes data redundancy Data is organized into separate tables for each entity, so it reduces data duplicates, saves storage space and ensures consistency.	Complexity in database schema Complex schema with multiple tables and relationships can make it difficult for developers to understand and maintain the database structure, especially in large or evolving systems.
Ensures data integrity Using specific rules regarding relationships and dependencies, normalization ensures insertion, update, delete queries and maintains data integrity.	Increased query complexity When needing to join multiple tables to perform queries, performance can drop.
Facilitates efficient data retrieval Well-defined relationships between tables support the development of efficient queries.	Increased storage requirements The increased number of tables and relationships may lead to higher storage requirements.
Supports scalability New data can be added without significantly altering the existing structure.	Not ideal for all use cases Normalization is based on relational database principles, and it may not fit all applications or types of data.

Advantages	Disadvantages
Promotes data consistency Eliminating data redundancy and defining clear relationships promotes data consistency.	Difficulty in balancing normalization When normalizing a database, you need to aim for a balance between reducing data redundancy and maintaining performance. Over-normalization (too many tables and relationships) or under-normalization (failing to separate data appropriately) may lead to maintenance issues and low performance.
Simplifies database maintenance Changes can be made to a single table without affecting other tables.	Overheads in updates Updating records may require changing data in different places, which can reduce performance or increase the complexity of update operations.

■ Denormalization

Denormalization refers to deliberately allowing for data redundancy in a database design to improve the performance of queries.

◆ **Denormalization:** deliberately allowing for data redundancy in a database design to improve the performance of queries.

Advantages	Disadvantages
Improved query performance There is less need for joins; the simpler structure of the tables improves the performance of read-heavy queries.	Data redundancy There is the possibility of inconsistencies if updates are not properly managed; it is more challenging to synchronize and maintain the data.
Simplified data retrieval Data is stored closer to the way it is accessed by applications so it allows for faster retrieval of data as there are less complex joins between tables.	Increased storage overhead Redundant data requires additional storage space, which can become significant in large or evolving systems.
Enhanced scalability This reduces the overhead of maintaining complex relationships, so it supports larger data sets and higher transaction volumes without sacrificing performance.	Maintenance challenges Managing denormalized databases requires careful planning and maintenance to ensure data integrity and consistency.

There are situations where denormalization can enhance performance, especially when read performance is crucial and outweighs concerns about data redundancy and updates complexity. Some specific scenarios are as follows:

Read-intensive applications

Read-intensive applications refer to applications in which the focus is on retrieving data, rather than updating it. By reducing the number of tables, relationships and joins, data can be accessed faster, improving query performance and response time.

Reporting and analytics

Reporting and analytics are used when generating complex reports or analysing large sets of data. Reducing the number of tables, simplifying relationships and decreasing the need for joins speeds up reporting and analytics queries.

Data warehousing

In data warehousing, the focus is on storing and analysing historical data from different sources. Simplifying complex queries across different data sources and using fewer tables to organize related data improves query performance and response time.

Denormalization simplifies query structures by reducing the need for joins between tables. Simpler queries are easier to write, faster to execute, easier to understand and easier to maintain. On the other side, allowing for data redundancy means increased storage

requirements, and increased complexity in maintenance and update operations. Therefore, there is a need to find a balance between the two, and this requires:

- analysing the specific requirements of the application (is the focus on reading or retrieving the data or on updating it?)
- identifying whether the benefits of denormalizing the database are higher than the risk of data redundancy and complexity (sometimes a partially denormalized schema might be a solution)
- implementing appropriate strategies to mitigate risks and optimize performance (implement robust data validation, monitor query performance and storage requirements, and so on).

ACTIVITY

Independent learning: Independent reflection and targets for improvement – identify your strongest and weakest points and set short-term goals for achieving success. When analysing your strongest and weakest points, consider: technical skills and soft skills; constructive or positive feedback from peers and teachers; successful projects; formative assessments; and knowledge gaps or technical gaps. Short-term goals will keep you focused; they will allow you to set an action plan that can be monitored and evaluated progressively.

Utilitarianism, the greatest good for the greatest number. The ends justify the means.

Utilitarianism is an ethical theory that suggests the best action is the one that maximizes overall happiness or well-being. This approach is often summarized by the phrase "the greatest good for the greatest number". According to this view, the moral value of an action is determined by its outcome or consequences, rather than by any intrinsic qualities of the action itself. For example, if sacrificing the well-being of a few individuals leads to a greater overall benefit for society, a utilitarian would argue that such a sacrifice is justified.

When managing databases, particularly those containing personal information, there's often a trade-off between privacy and utility. A utilitarian approach might justify the use of personal data without explicit consent if it leads to a greater good, such as improving public health through data-driven insights. During a pandemic, health authorities might access and analyse large data sets of personal health information to track the spread of the virus. From a utilitarian perspective, the potential benefits to society (for example, controlling the pandemic) might outweigh individual privacy concerns. The utilitarian principle of "the greatest good for the greatest number" provides a framework for making decisions about how databases are managed, used and secured.

Discuss how this approach may be balanced against other ethical considerations, such as individual rights and fairness, which may not always align with a purely utilitarian perspective.

REVIEW QUESTIONS

- 1 Define the term "schema".
- 2 Discuss the characteristics of a normalized database.
- 3 Identify two issues caused by data redundancy.
- 4 State the characteristics of:
 - a 1NF
 - b 2NF
 - c 3NF
- 5 Distinguish between conceptual and logical schema.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.3.1 Outline the difference between data language types within structured query language (SQL)
- ▶ A3.3.2 Construct queries between two tables in SQL
- ▶ A3.3.3 Explain how SQL can be used to update data in a database
- ▶ A3.3.4 Construct calculations within a database using SQL's aggregate functions (HL)
- ▶ A3.3.5 Describe different database views (HL)
- ▶ A3.3.6 Describe how transactions maintain data integrity in a database (HL)

A3.3.1 Data language types within SQL

Data language types include data definition language (DDL) and data manipulation language (DML).

■ Data definition language (DDL)

Data definition language (DDL) is used to create, modify and remove data structures from a relational database.

◆ **Data definition language:** language that is used to create, modify and remove data structures from a relational database.

SQL DDL instructions	Explanation
CREATE	Create a new database object (table, view, index)
PRIMARY KEY	Set a field as a primary key
FOREIGN KEY ... REFERENCES ...	Set a field as a foreign key by specifying the field and the table it is associated with
ALTER	Change the structure of an existing database object: alter a table structure, by adding or removing columns or adding constraints
DROP	Delete database objects (tables, indices, views)

CREATE statements

CREATE DATABASE:

```
CREATE DATABASE HOSPITAL
```

CREATE TABLE:

```
CREATE TABLE Employees (
  EmployeeID INT,
  DepartmentID INT PRIMARY KEY,
  LastName VARCHAR(20)
);
```


CREATE VIEW:

```
CREATE VIEW EmployeeDetails AS
SELECT EmployeeID, LastName
FROM Employees
JOIN Departments ON Employees.DepartmentID = Departments.
DepartmentID;
```

CREATE INDEX:

```
CREATE INDEX idx_last_name ON Employees(LastName);
```

ALTER statements

Add primary key:

```
ALTER TABLE Employees
ADD PRIMARY KEY (EmployeeID);
```

Add foreign key:

```
ALTER TABLE Employees
ADD FOREIGN KEY DepartmentID REFERENCES Department(DepartmentID);
```

Add a column:

```
ALTER TABLE Employees
ADD Email VARCHAR(25);
```

Drop a column:

```
ALTER TABLE Employees
DROP COLUMN Email;
```

Add a constraint:

```
ALTER TABLE Employees
ADD CONSTRAINT FK_DepartmentID
FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID);
```

DROP statements

Drop table:

```
DROP TABLE Employees;
```

Drop index:

```
DROP INDEX idx_last_name;
```

Drop view:

```
DROP VIEW EmployeeData;
```

■ Data manipulation language (DML)

Data manipulation languages are used to add, modify, delete and retrieve data stored in relational databases.

◆ **Data manipulation language:** language that is used to add, modify, delete and retrieve data stored in relational databases.

SQL DML instructions	Explanation
SELECT	Retrieves data from one or more tables
INSERT	Adds records into a table
DELETE	Removes records from a table
UPDATE	Modifies existing records in a table

SELECT statements

Retrieve records by displaying specific columns:

```
SELECT field1, field2, field3...  
FROM table_name;
```

Retrieve records by displaying attributes that match a given criterion:

```
SELECT field1, field2, ...  
FROM table_name  
WHERE condition;
```

Retrieve all records:

```
SELECT * FROM table_name;
```

Retrieve records by checking whether specific fields meet specific criteria:

```
SELECT * FROM table_name WHERE condition;
```

A3.3.2 SQL queries between two tables

Including a JOIN in a SELECT statement allows you to aggregate data from multiple tables. For example, considering the employees table and the department table, the script below retrieves the salary expense grouped by department.

■ JOIN in a SELECT statement

```
SELECT Employees.DepartmentName, SUM(Employees.Salary) AS TotalSalary  
FROM Employees  
JOIN Department ON Employees.DepartmentID = Department.DepartmentID  
GROUP BY Department.DepartmentName;
```

■ DISTINCT in a SELECT statement

When you want to retrieve unique records and ignore duplicates, you can use the keyword DISTINCT.

```
SELECT DISTINCT column1, column2, ...  
FROM table_name;
```

■ HAVING clause vs WHERE clause

The HAVING clause is used to filter groups of records created by the GROUP BY clause, for example when needing to retrieve the department ID and the average salary per department where the average salary is above 10,000.

The WHERE clause is used to filter records *before* grouping, while the HAVING clause is used to filter groups *after* aggregation.

```
SELECT DepartmentID, Salary  
FROM Employees  
GROUP BY DepartmentID  
HAVING Salary > 10000;
```

■ RELATIONAL operators

Relational operators can be used to fetch data that meets specific criteria.

Operator	Example
Equals to	<code>SELECT * FROM Employees WHERE DepartmentID = 1;</code>
Not equals to	<code>SELECT * FROM Employees WHERE DepartmentID <> 1;</code>
Greater than	<code>SELECT * FROM Employees WHERE Salary > 10000;</code>
Smaller than	<code>SELECT * FROM Employees WHERE Salary < 10000;</code>
Greater than or equals to	<code>SELECT * FROM Employees WHERE Salary >= 10000;</code>
Smaller than or equals to	<code>SELECT * FROM Employees WHERE Salary <= 10000;</code>

■ FILTERING

Operator	Example
BETWEEN filters values between a range	<code>SELECT * FROM Employees WHERE Salary BETWEEN 50000 AND 100000;</code>
IN filters values that match any value in a given list	<code>SELECT * FROM Employees WHERE DepartmentID IN (1, 2, 3);</code>
IS NULL filters records with null values	<code>SELECT * FROM Employees WHERE ManagerID IS NULL;</code>
IS NOT NULL filters records with non-null values	<code>SELECT * FROM Employees WHERE ManagerID IS NOT NULL;</code>
Combining conditions using logic operators	<code>SELECT * FROM Employees WHERE (DepartmentID = 5 AND Salary > 50000) OR (DepartmentID = 1 AND Salary > 10000);</code>

■ Pattern matching

- LIKE filters values based on a pattern.
- % is used for any sequence of characters (zero or more).
- _ is used for one single character.

This will retrieve records that start with the letter D:

```
SELECT * FROM Employees  
WHERE LastName LIKE 'D%';
```

This will retrieve records that end with the letter d:

```
SELECT * FROM Employees  
WHERE LastName LIKE '%d';
```

This will retrieve records that match a specific pattern (start with the letter D, followed by a character, followed by the letter m and followed by zero or more characters):

```
SELECT * FROM Employees  
WHERE LastName LIKE 'D_m%';
```

This will retrieve records that are made of three characters:

```
SELECT * FROM Employees  
WHERE LastName LIKE ' _ _ ';
```

■ Ordering data

Ordering by a single field:

```
SELECT * FROM Employees  
ORDER BY LastName;
```

Ordering by a single field in ascending order:

```
SELECT * FROM Employees  
ORDER BY LastName ASC;
```

Ordering by a single field in descending order:

```
SELECT * FROM Employees  
ORDER BY LastName DESC;
```

Ordering by multiple fields:

```
SELECT * FROM Employees  
ORDER BY DepartmentID, Salary DESC;
```

A3.3.3 SQL update queries

SQL statement	Explanation	Example
INSERT	Adds new records	INSERT INTO table_name (field1, field2, ...) VALUES (value1, value2, ...);
DELETE	Deletes records	DELETE FROM table_name WHERE condition;
UPDATE	Modifies records	UPDATE table_name SET field1 = value1, field2 = value2, ... WHERE condition;

Indexed columns optimize query performance, but updating data in indexed columns may impact performance. When an update is performed for an indexed column, the index needs to be updated as well, which can slow down the operation. Also, index updates can lead to blocking other transactions that need access to a specific record. To overcome those challenges, you can batch the updates to reduce the number of times the index needs to be changed; frequently rebuild or reorganize indexes to reduce fragmentation; or use partial indexes or filtered indexes to limit the scope of the index only to the most relevant records.

A3.3.4 SQL's aggregate functions (HL)

Aggregate functions are used to perform calculations on multiple records based on a given field. Such functions are AVERAGE, COUNT, MAX, MIN, SUM.

Aggregate function	Example
SUM returns the total value of a numerical field	SELECT SUM(Salary) FROM Employees;
COUNT returns the number of records that meet the given criteria	SELECT COUNT(EmployeeID) FROM Employees;
AVERAGE returns the average value of a numerical field	SELECT AVG(Salary) FROM Employees;
MIN returns the smallest value in a field	SELECT MIN(Salary) FROM Employees;
MAX returns the largest value in a field	SELECT MAX(Salary) FROM Employees;

◆ **Aggregate functions:** functions used to perform calculations on multiple records based on a given field, e.g. AVERAGE, COUNT, MAX, MIN, SUM.

■ Aggregate functions on grouped data

Using aggregate functions on grouped data aids reporting and decision-making. An example would be to display the number of employees for each department:

```
SELECT Department.DepartmentName, COUNT(Employees.EmployeeID)
AS ECount
FROM Employees
JOIN Department ON Department.DepartmentID = Employees.DepartmentID
GROUP BY Department.DepartmentName;
```

Another example is when you want to display the average salary for each department:

```
SELECT Department.DepartmentName, AVG(Employees.Salary) AS AvgSalary
FROM Employees
JOIN Department ON Employees.DepartmentID = Department.DepartmentID
GROUP BY Department.DepartmentName;
```

And yet another example would be to display the minimum and maximum salary per department:

```
SELECT DepartmentID, MIN(Salary) AS MinSal, MAX(Salary) AS MaxSal
FROM Employees
GROUP BY DepartmentID;
```

◆ **View:** a virtual table based on the result set of a SELECT query. They do not store data themselves but provide a way to present the data from one or more tables in a customized manner.

A3.3.5 Database views (HL)

A **view** is a virtual table based on the result set of a SELECT query. They do not store data themselves, but provide a way to present the data from one or more tables in a customized manner. Multiple views present different subsets of the data to different users, with the data being presented in different ways according to the user's needs.

There are several advantages of using views:

- **Data complexity hiding:** Views can encapsulate complex queries, simplifying the process for users to query data without needing to understand the underlying SQL.
- **Data consistency:** Views can present data in a consistent manner, even if the underlying tables are modified.
- **Data independence:** The database schema can be changed without affecting the user views.
- **Performance:** Views can increase performance by simplifying complex queries, by abstracting join operations into a single reusable object (the view).
- **Query simplification:** Queries can be simplified by breaking them down into smaller parts, hiding unnecessary details, applying filters and calculations and displaying the results in a view.
- **Read-only or updatable views:** When updating a view, the changes are passed through to the underlying tables from which the view was created, only if certain conditions are met. If those conditions are met, the view is updatable; otherwise, it is read-only. There are three conditions for a view to be updatable: it must be a subset of a single table or another updatable view; all base table fields excluded from the view definition should allow NULL values; and the SELECT statement of the view should not contain sub-queries (a DISTINCT predicate, a HAVING clause, aggregate functions, joined tables, user-defined functions or stored procedures).

- **Security:** Views can limit access to specific attributes or records, providing a way to control which data users have access to. There are different types of database views. Some of those types are simple views, complex views or materialized (snapshot) views.

■ Simple views

Simple views are views based on a single table that do not include complex queries, such as aggregate functions or joins.

An example of a simple view is when displaying some fields from a table. In this case, three fields (EmployeeID, FirstName and LastName) are displayed from the Employees table.

```
CREATE VIEW EmpNames AS
SELECT EmployeeID, FirstName, LastName
FROM Employees;
```

■ Complex views

Complex views are views that include multiple tables and complex queries, such as aggregate functions or joins.

An example of a complex view would be to display data from the Employees and Department tables.

```
CREATE VIEW EmpDepartment AS
SELECT Employees.EmployeeID, Employees.LastName,
       Department.DepartmentName
FROM Employees
JOIN Department ON Employees.DepartmentID = Department.DepartmentID;
```

■ Materialized (snapshot) views

Materialized views are views that can be frequently refreshed that store pre-computed data sets derived from a SELECT query and stored for later use. As it avoids query re-run (which is used in regular views), it often delivers data faster. The code below is an example of a view that displays the employee ID and their salary for each employee in the Employees table.

Creating the snapshot view:

```
CREATE MATERIALIZED VIEW TotalSalaries AS
SELECT EmployeeID, SUM(Salary) AS TotalSal
FROM Employees
GROUP BY EmployeeID;
```

Querying the snapshot view:

```
SELECT * FROM TotalSalaries;
```

A3.3.6 How transactions maintain data integrity in a database (HL)

■ ACID

Transactions are sequences of SQL operations that are executed as a single unit of work, ensuring data integrity and consistency.

ACID is an acronym that refers to the four properties that define a transaction:

■ Atomicity

Transactions are atomic (indivisible and treated as a whole). Either all the actions within a transaction are completed successfully, or none of them is. If any part of the transaction fails, the entire transaction is rolled back, and the database remains unaffected.

■ Consistency

Transactions ensure the data follows predefined rules or constraints. The database must be in a valid and expected state after the completion of a transaction.

■ Isolation

Transactions are isolated from each other to prevent interference. This ensures that concurrent execution of multiple transactions does not lead to data inconsistencies.

■ Durability

Once a transaction is committed and completed successfully, its changes are permanent. Even if the system fails, the changes made by a committed transaction are preserved. Durability is important because transaction data changes must be available even if the database is failing.

● Top tip!

When approaching exam questions, ensure you make efficient use of terminology. Often, candidates seem to understand the concepts, but they provide generic responses, with their answers often lacking precision. Terminology must be used precisely when writing responses to gain full marks.

■ Transaction control language (TCL) commands

TCL commands in SQL are used to manage the transactions. Some of those commands are:

■ **BEGIN TRANSACTION:** Used to start a new transaction in SQL.

■ **COMMIT:** Used to save all changes made during the current transaction to the database. Once this operation is performed, the changes are permanent and visible to other users.

```
BEGIN TRANSACTION;
```

```
UPDATE Employees SET Salary = Salary + 100 WHERE EmployeeID = 1;
```

```
UPDATE Employees SET Salary = Salary - 100 WHERE EmployeeID = 5;
```

```
COMMIT;
```

In the example above, the UPDATE statements for the two records are written and they are both saved once the COMMIT statement is reached.

- **ROLLBACK:** Used to undo all changes made during the current transaction. It reverts the database to the state it was in before the transaction began.

In the example below, the two `UPDATE` statements are reversed, and so the table would reach its state prior to the updates being made.

```
BEGIN TRANSACTION;
UPDATE Employees SET Salary = Salary + 100 WHERE EmployeeID = 1;
UPDATE Employees SET Salary = Salary - 100 WHERE EmployeeID = 5;
ROLLBACK;
```

TOK

How has the development of database technology influenced the way we acquire and process knowledge?

The development of database technology has profoundly influenced the ways in which we acquire and process knowledge, touching on key areas such as the nature of knowledge, how it is shared and the ethical considerations involved.

Databases store vast amounts of data, but this raw data only becomes useful when it is processed into information and then interpreted as knowledge. This raises questions about the nature of knowledge itself. How do we distinguish between data, information and knowledge? How does the structure of a database influence what we consider to be true or valuable knowledge?

Databases rely on structured query languages (SQL) and other forms of data communication. The precision and clarity required in database queries contrast with the ambiguity and richness of natural language. This might lead to a more structured, but potentially limited, way of knowing. How does the structured nature of database queries influence our understanding of complex or ambiguous information?

REVIEW QUESTIONS

- 1 Discuss whether a view is physically stored in a database.
- 2 Define the term “database transaction”.
- 3 Identify a reason a transaction may need to be rolled back by giving an example.
- 4 State the effect of rolling back a transaction.
- 5 Describe the four properties that describe a transaction.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.4.1 Outline the different types of databases as approaches to storing data
- ▶ A3.4.2 Explain the primary objectives of data warehouses in data management and business intelligence
- ▶ A3.4.3 Explain the role of online analytical processing (OLAP) and data mining for business
- ▶ A3.4.4 Describe the features of distributed databases

A3.4.1 Types of databases as approaches to storing data

Database models represent frameworks that determine the logical structure of a database and influence how data is stored, organized and manipulated. Different database models cater to different types of applications and requirements.

■ NoSQL databases

◆ **NoSQL database:** a database designed to handle large volumes of data and diverse data types, structured differently from relational databases.

NoSQL databases are designed to handle large volumes of data and diverse data types. They offer flexibility and scalability, making them suitable for modern web applications, such as e-commerce platforms, big data and real-time analytics.

NoSQL databases store data differently from relational databases. There are four main types of NoSQL databases: document databases, key-value databases, wide-column store databases and graph databases.

Document databases

In document databases, data is stored in documents like JSON (JavaScript Object Notation) objects. The documents contain pairs of fields and values, and document databases are used for content management systems and e-commerce platforms. They offer a flexible data model, suitable for semi-structured and unstructured data sets. They provide an easy way to represent hierarchical data, but a disadvantage is that they pose a risk of data redundancy and inconsistency. An example of such a database is MongoDB. Here is an example:

```
{
  "_id": "12345",
  "name": "blabla",
  "email": "blabla@car.com",
  "address": {
    "street": "bloblo street",
    "city": "omega city",
  },
  "services": ["transport", "tourism"]
}
```


Key-value databases

In key-value databases, data is stored as key-value pairs. Key-value databases are used for real-time analytics, caching and session management. They are simple databases that allow for fast read and write operations; however, they have limited querying capabilities. An example of a key-value database is Redis. Here is an example:

Key: user:12345

Value: {"name": "blabla", "email": "blabla@car.com", "job": "transporter"}

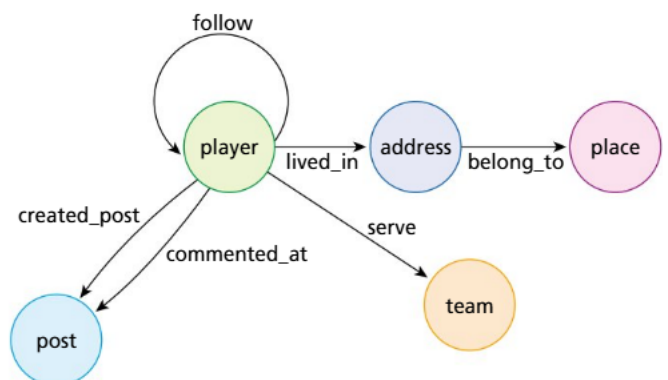
Wide-column store databases

In wide-column store databases, data is stored in tables, rows and dynamic columns. Different rows can have different sets of columns. They enable efficient retrieval of sparse and wide data and are used in big-data applications or real-time analytics. They are proven to be efficient for read and write operations on large data sets and are easy to scale horizontally; however, this is a complex model to implement. An example of a wide-column store database is Cassandra. Here is an example:

name	id	email	dateOfBirth
blabla	12345	blabla@car.com	
nathan	1234		12-12-2000

Graph databases

In graph databases, data is stored as nodes and edges in a graph structure. Nodes usually store data about people, places or things, and edges store data about the relationships between nodes. They are used for social-media platforms or recommendation engines. Graph databases are great for representing and querying complex relationships, but they use a specialized query language, and it is more complex to maintain and optimize them. An example of a graph database is OrientDB. Here are two examples of graph databases:



◆ **Cloud database:** a database that runs on cloud computing platforms, providing scalability, high availability and flexible resource management.

◆ **Spatial database:** a database optimized to store and query data related to objects in space, including points, lines and polygons.

◆ **In-memory database:** a database that stores data entirely in the main memory (RAM) rather than on disk, providing extremely fast read and write operations.

■ Cloud databases

Cloud databases are databases that run on cloud computing platforms, providing scalability, high availability and flexible resource management. They can be NoSQL or SQL databases.

In cloud databases, pricing is based on the use of system resources, which can be provisioned on demand as needed to meet processing workloads. Organizations can choose between two models when they opt for cloud databases:

- **Self-managed database:** An infrastructure as a service (IaaS) environment, in which the database runs on a virtual machine on a system operated by a cloud provider. The provider manages and supports the cloud infrastructure, including servers, operating systems and storage devices. But the organization is responsible for database deployment, administration and maintenance, so it has full control over the database.
- **Managed database services:** Fully managed by the vendor, both the system infrastructure and the database platform are managed for the customer; the vendor handles provisioning, backups, scaling, patching, upgrades and other basic database administration functions; organizations monitor the database, and they can collaborate with the vendor on some administrative functions.

Examples of database-as-a-service (DBaaS) are Amazon DynamoDB and Azure Cosmos DB. They are used for startups, IoT or web-scale applications; they are fully managed, offer pay-as-you-go pricing and the organization has limited control over the infrastructure.

Examples of managed databases are Amazon RDS, Google Cloud SQL and Azure SQL database. They are used for web and enterprise applications, and they depend on cloud providers and potentially have higher costs.

■ Spatial databases

Spatial databases are optimized to store and query data related to objects in space, including points, lines, polygons, 3D shapes and coordinates. They support spatial data types and spatial indexes to access the data, and they support geometric functions. They are used for Geographic Information Systems (GIS), location-based services and mapping applications. Examples are: Oracle Spatial, PostGis and MongoDB with Geospatial Indexes. Such models efficiently store and query spatial data, but they use complex data types and queries that require specialized knowledge or experts.

■ In-memory databases

In-memory databases store data entirely in the main memory (RAM) rather than on disk, providing extremely fast read and write operations. They are used for real-time analytics, caching and gaming applications. They allow for extremely fast data access and transaction processing, but they have low latency for read and write operations. Examples are Redis and Oracle TimesTen.

A3.4.2 Primary objectives of data warehouses in data management and business intelligence

◆ **Data warehouse:** a specialized type of database designed for analytical purposes rather than transactional processing.

A **data warehouse** is a specialized type of database designed for analytical purposes rather than transactional processing. It is used to store and analyse large volumes of historical data from various sources to support decision-making processes within organizations. A data warehouse represents a repository of stored data related to a specific subject. It includes tools to extract, transform and load data into the repository and tools to manage and retrieve the metadata.

The characteristics of data warehouses are discussed below:

■ Append-only

This characteristic means that, when data is loaded using append-only, existing records are not updated, but instead are appended to tables as new rows. Therefore, at a later stage, the tables will contain different versions of the records, so that how they changed over time can be analysed.

■ Subject-oriented

Data warehouses are organized around key subjects or themes relevant to the organization, such as sales, marketing or finance. They help with organizing and presenting data in a way that is aligned with the analytical needs and objectives of the organization.

■ Integrated data

Data from multiple operational systems and external sources are integrated into a single repository using ETL (**extract, transform, load**) processes. This consolidates information from multiple sources into a centralized repository, providing a single source of truth for the organization. When ETL is carried out, certain precautions should be taken, such as:

- ensuring the extraction does not affect the source system
- ensuring the extracted data can be read by the current system
- ensuring the different data formats being extracted can be converted to become readable by the system and can be formatted
- ensuring that the data is relevant to what the user wants to extract and utilize.

■ Time-variant

Data warehouses store historical data to support time-based analysis, enabling comparisons and trend analysis over time. Data warehousing is time-dependent because the content in the data warehouse is only valid for a period, because the data undergoes changes dynamically, and its focus on change over time is time-variant.

■ Non-volatile

Data once loaded into the data warehouse is rarely updated or deleted, ensuring data integrity and consistency for analytical purposes.

■ Optimized for query performance

Data warehouses seek to determine the most efficient way to execute a given query by considering a variety of query execution strategies. It directly impacts the speed and efficiency of data retrieval and analysis processes.

A3.4.3 The role of OLAP and data mining for business intelligence

Business intelligence refers to the technologies, applications and practices for collecting, integrating, analysing and presenting business information. Its aim is to support data-driven decision-making and improve business performance.

Online analytical processing (OLAP) and **data mining** are technologies used for data analysis and business intelligence, enabling organizations to extract valuable insights and make informed decisions from their data.

◆ **Extract:** to gather data from various operational databases, flat files, APIs, etc.

◆ **Transform:** to aggregate and transform data into a consistent format suitable for analysis.

◆ **Load:** to load transformed data into a data warehouse.

◆ **Business intelligence:** technologies, applications and practices for collecting, integrating, analysing and presenting business information.

◆ **Online analytical processing:** the software technology you can use to analyse business data from different points of view.

◆ **Data mining:** the process of sorting through large data sets to identify patterns and relationships that can help solve business problems through data analysis.

■ Role of OLAP in business intelligence

OLAP facilitates interactive analysis of multidimensional data, allowing users to explore data from various dimensions, such as time, product or region. It provides pre-aggregated views of data, which are optimized for querying and reporting, allowing for quick retrieval of summarized information, and supports decision-making processes. OLAP supports complex calculations and analytical functions directly on aggregated data, such as year-over-year comparisons. It supports data visualization techniques, such as charts, graphs and dashboards, and users can create ad hoc queries to explore data dynamically and answer specific business questions without needing to rely on predefined reports.

■ Role of data mining in business intelligence

Data mining involves discovering patterns and relationships within large data sets, using statistical algorithms, machine learning techniques and artificial intelligence to uncover hidden insights. It enables predictive modelling by analysing historical data to forecast future trends and outcomes, which helps with predicting customer behaviour, demand forecasting and risk assessment. As such, it helps in customer segmentation based on attributes and behaviours, by identifying customer segments with similar characteristics for targeted marketing campaigns and personalized customer experiences. It can detect anomalies in data, which may indicate fraud, errors or unusual patterns that require further investigation. Database segmentation can help to increase the profit of the organization, increase its reputation, increase the number of customers and provide better opportunities for growth.

Data mining techniques include the following:

- **Classification:**
 - A supervised learning technique that categorizes data into labels based on input features.
 - Used in spam email detection, sentiment analysis and credit scoring.
- **Clustering:**
 - An unsupervised learning technique that groups similar data points together into clusters based on their characteristics or proximity in feature space.
 - Finds patterns in customer behaviour by grouping and analysing variables to connect them; it can find previously unknown links that help in decision-making.
 - Used in market segmentation, customer profiling and anomaly detection.
- **Regression:**
 - Predicts continuous numerical values based on input variables, aiming to establish relationships between variables.
 - Used in sales forecasting, price predictions and risk assessment.
- **Association rule discovery:**
 - Identifies relationships or associations between items in large data sets, typically in transactional databases.
 - Looks at how entities or events are connected, and finds where one or more events may lead to another.
 - Correlates the presence of a set of items with another range of values for another set of variables, breaking up the data sets by variables such as location, age, gender.
 - Used in cross-selling recommendations.
- **Sequential pattern discovery:**
 - Discovers patterns or sequences in data, where events occur in a specific order, over a specific period of time (temporal patterns).
 - Used in web log analysis and clickstream analysis.

- **Anomaly detection:**

- Identifies rare or unusual patterns in data that do not conform to expected behaviour.
- Used in fraud detection, network security monitoring and equipment failure prediction.

◆ **Distributed database:** a database made of two or more files located on different sites on the same network or on completely different networks.

A3.4.4 Features of distributed databases

A **distributed database** is a database made of two or more files located on different sites on the same network or on completely different networks. Although they are stored on different sites or different computer systems, they provide a fully functional, unified view of data to users and applications.

Distributed databases are used in different areas, such as online retailers using them to manage product catalogues, inventory and transactions across distributed warehouses, and telecom companies using them to manage subscriptions, network traffic analysis and service provisioning across different geographical locations.

Maintaining data consistency in a distributed system is crucial for ensuring the reliability, accuracy and trustworthiness of data across all nodes and users. This helps with reliable decision-making; avoiding data corruption; maintaining data integrity; and increasing user satisfaction and trust.

The role of atomicity, consistency, isolation and durability (ACID) to ensure reliable processing of transactions in distributed databases is as follows:

- **Atomicity**

Atomicity ensures that distributed transactions are either committed across all nodes or rolled back completely if any part of the transaction fails. This prevents partial updates and maintains data consistency across nodes.

- **Consistency**

Consistency in distributed systems ensures that all nodes have access to the same consistent view of data after a transaction is completed.

- **Isolation**

Isolation prevents interference between concurrent transactions executing on different nodes. It prevents the modification of the same data item by two different transactions.

- **Durability**

Durability ensures that committed transactions are reliably stored and replicated across distributed nodes.

The features of distributed databases are described below:

Concurrency control

Concurrency control refers to techniques used to manage simultaneous access and modifications to shared data across multiple nodes in a distributed database. It ensures that transactions execute correctly and maintain consistency, despite potential conflicts that may arise due to concurrent operations. Locking mechanisms can be used to enforce isolation and prevent conflicting operations, or unique timestamps can be assigned to each transaction to determine the order of execution. For example, different systems may attempt to access the same data at the same time, such as two systems attempting to update the same piece of data. If one starts the update and then the second finishes before the first is saved,

this could potentially lead to inconsistent updates. In such cases, the solution is to isolate the transactions; when one system is accessing the data, that transaction is locked, and it is released only after the transaction is committed.

Data consistency

All copies of data across distributed nodes are synchronized and reflect the most recent, correct state of information. Strong consistency models ensure that all updates to data are visible to all nodes immediately after they occur.

Data partitioning

Data partitioning improves performance, scalability and manageability by distributing data across multiple nodes or servers. It allows databases to handle large volumes of data and high transaction rates efficiently.

Data security

By implementing robust security measures and continuously monitoring for threats and vulnerabilities, organizations can mitigate risks and safeguard sensitive data across distributed environments effectively. To achieve this, organizations can:

- encrypt sensitive data stored on disks or databases to protect against unauthorized access
- define roles and permissions that restrict access to data based on users' roles
- use strong authentication mechanisms (multifactor authentication) to verify user identities before granting access to the database.

Distribution transparency

Distribution transparency refers to the ability to access and manipulate data across multiple nodes or servers in a transparent and seamless manner.

Fault tolerance

Fault tolerance refers to systems' ability to continue operating and providing services even in the presence of hardware failures, software errors or network disruptions, by using data-redundancy systems, monitoring and recovery in case of failure detection.

Global query processing

Global query processing involves the coordination and execution of queries that span multiple distributed nodes or databases.

Location transparency

Location transparency refers to the ability of a system to hide the physical or logical location of resources and services (network address or server details) from users and applications. It ensures that users can access resources or services without being aware of their specific location, simplifying system management.

Replication

Replication addresses the creation and maintenance of copies of data across multiple nodes, servers or locations, which enhances data availability, reliability and performance.

Scalability

Scalability refers to the ability of the database system to handle increasing amounts of data and user requests by efficiently distributing workload across multiple nodes or servers. It ensures the database can grow to meet performance and capacity requirements as demands increase.

ACTIVITY

Social skills: Listen actively to other perspectives and ideas – there are different ways to solve a problem, some better than others; listen to advice and try new techniques and problem-solving strategies. In any collaborative environment, actively listening to the perspectives and ideas of others is crucial for effective problem-solving and decision-making. Different individuals bring diverse experiences, viewpoints and approaches to the table, each offering unique insights into how a problem can be addressed. By carefully listening to others, you not only gain a broader understanding of the problem, but you also become aware of innovative solutions that you might not have considered on your own.

REVIEW QUESTIONS

- 1 Define the term “data mining”.
- 2 Define the term “data warehouse”.
- 3 Describe how regression is used in data mining.
- 4 Describe the concurrency control feature in distributed databases. Explain how this can be achieved.
- 5 Explain the role of integrated data in a data warehouse.
- 6 Outline two methods to ensure the security of a data warehouse.
- 7 Compare classification and sequential pattern discovery in data mining.
- 8 Outline the differences between a database view and a data warehouse.

 Linking questions

- 1 What processes are needed to store data in database structures so that they can be used in machine learning? (A4)
- 2 How does database programming in SQL differ from programming computationally in a high-level language? (B2)
- 3 To what extent is the effectiveness of the distributed database determined by the network that connects the various tables? (A2)
- 4 How could machine learning be applied to databases? (A4)
- 5 How do programming languages interact with databases to store, retrieve and manipulate data? (B2)

EXAM PRACTICE QUESTIONS

- 1 A telecommunications company is designing a relational database to store its desk tickets. The database will have the following tables:
- ```
Operator(OperatorID, OperatorName, Location)
Engineer(EngineerID, EngFirstName, EngLastName, EngLocation, Salary)
Ticket(TicketNo, Location, Status, OperatorID, TicketDate, TicketPriority)
Supplier(SupplierID, Email, PhoneNo)
Product(ProductID, ProductName, Price, SupplierID)
ProductOrder(TicketNo, OrderID, ProductID, Quantity, EngineerID)
```
- Identify the primary key for each of the tables described above. [3]
  - Identify three benefits of a relational database. [3]
  - Construct an ERD for the relational database. [3]
  - Construct the SQL DDL instructions to create the `Ticket` table. [1]
  - Construct the SQL DML statements to return the total product quantity ordered by the engineer with the engineer ID D893. [4]
  - Identify a foreign key in the `Product` table. [1]
  - State whether the database is normalized and whether it is in third normal form (3NF). [1]
  - Describe the characteristics of a database that is in third normal form (3NF). [3]
  - The following table is an example of the `Engineer` table.

| EngineerID | EngFirstName | EngLastName | EngLocation | Salary |
|------------|--------------|-------------|-------------|--------|
| D893       | Daniel       | Buchidau    | Trier       | 7000   |
| D894       | Constantin   | Constantin  | Heidelberg  | 6000   |
| D895       | Martin       | Bond        | Cologne     | 6500   |

- Define the term "tuple". Give an example of a tuple from the `Engineer` table. [2]
  - State the number of fields in the `Engineer` table. [1]
  - Construct an SQL statement to increase the salary of the engineer with the ID D894 by 300. [2]
  - Construct SQL statements to update the `Engineer` table to include two more fields: one called `Experience`, to store how many years of experience each engineer has, and one called `IncreaseDate`, which includes the date when the last salary raise occurred. [4]
  - Construct an SQL script to find the average salary in the `Engineer` table. [2]
- State what a transaction is and identify the four properties that define a transaction. [2]
  - Identify three advantages of using views in a database. [3]
  - Explain three data mining techniques. [6]
  - Compare cloud and spatial databases. [4]
  - Describe two features of distributed databases. [2]
  - Identify and compare a NoSQL database with a relational database. [4]
  - Explain the role of online analytical processing (OLAP) in business intelligence. [3]
  - Explain the COMMIT transaction control language (TCL) command. [1]
  - Other than COMMIT, identify another transaction control language (TCL) command. [1]

